

WEEK-8

Task-1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

Code:

```
GNU nano 8.7.1 combined
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    fd = open("combined.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);

    // Redirect stderr to the same file
    dup2(STDOUT_FILENO, STDERR_FILENO);

    close(fd);

    printf("This is normal output.\n");
    fprintf(stderr, "This is error output.\n");
    printf("Output after error.\n");

    return 0;
}
```

Output:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# nano combined.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# gcc combined.c -o combined
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# ./combined
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# cat combined.txt
This is error output.
This is normal output.
Output after error.
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# |
```

Report: The combined redirection program was successfully implemented in Ubuntu.
Standard output and standard error were redirected to the same file using dup2().
Both normal and error messages were successfully stored and verified.
The ordering of the generated output was also checked.

Task-2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order,
Validate Syntax, Generate Execution Plans, Test Scenarios

Code:

```
GNU nano 8.7.1                                complex.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

int main() {
    int fd[2];
    pipe(fd);

    // First command: ls
    if (fork() == 0) {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", NULL);
        exit(1);
    }

    // Second command: grep .c > result.txt
    if (fork() == 0) {
        int file = open("result.txt",
                        O_WRONLY | O_CREAT | O_TRUNC,
                        0644);

        dup2(fd[0], STDIN_FILENO);
        dup2(file, STDOUT_FILENO);

        close(file);
        close(fd[0]);
        close(fd[1]);

        execlp("grep", "grep", ".c", NULL);
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    printf("Command executed successfully.\n");

    return 0;
}
```

Output:

```
Output of the program:
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# nano complex.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# gcc complex.c -o complex
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# ./complex
Command executed successfully.
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# cat result.txt
combined
combined.c
combined.txt
complex
complex.c
result.txt
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# |
```

Report:

The complex command execution was successfully implemented using pipes and redirection.

The output of one command was passed through a pipe to another command.

The final output was redirected to a file using `dup2()`.

The execution order and generated output were verified successfully.

Task-3: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

Code:

```
GNU nano 8.7.1 background.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Background process started. PID = %d\n", getpid());

        sleep(5);

        printf("Background process completed.\n");
        exit(0);
    }

    printf("Parent continues immediately.\n");
    printf("Background PID = %d\n", pid);

    sleep(2);

    printf("Parent is still running...\n");

    waitpid(pid, NULL, 0);

    printf("Background job finished.\n");

    return 0;
}
```

Output:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# nano background.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# gcc background.c -o background
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# ./background
Parent continues immediately.
Background PID = 761
Background process started. PID = 761
Parent is still running...
Background process completed.
Background job finished.
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# |
```

Report: The background job program was successfully implemented in Ubuntu. A child process was launched while the parent process continued its execution. The process ID was stored and the job status was monitored using `waitpid()`. This demonstrated the basic behavior of non-blocking background execution.

Task-4: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

Code:

```
pavani@DESKTOP-38OLEIN: ~/skill
GNU nano 8.7.1                                jobtable.c *
int id;
pid_t pid;
char state[20];
};

int main() {
    struct Job jobs[MAX_JOBS];
    int count = 0;

    pid_t pid = fork();

    if (pid == 0) {
        sleep(3);
        exit(0);
    }

    // Add job
    jobs[count].id = count + 1;
    jobs[count].pid = pid;
    snprintf(jobs[count].state,
             sizeof(jobs[count].state),
             "Running");

    count++;

    printf("Job Table:\n");
    printf("Job ID\tPID\tState\n");

    for (int i = 0; i < count; i++)
        printf("%d\t%d\t%s\n",
               jobs[i].id,
               jobs[i].pid,
               jobs[i].state);

    sleep(4);

    // Check process status
    if (waitpid(pid, NULL, WNOHANG) == 0) {
        printf("\nJob is still running.\n");
    } else {
        printf("\nJob completed.\n");
        count--;
    }

    printf("\nRemaining jobs: %d\n", count);

    return 0;
}
```

Output:

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# nano jobtable.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# gcc jobtable.c -o jobtable
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# ./jobtable
Job Table:
Job ID  PID      State
1       807      Running

Job completed.

Remaining jobs: 0
root@DESKTOP-QU0HVJI:~/SKILL/skill-8# |
```

Report: The job table was successfully created to store process IDs, job IDs, and their states.

The program tracks the running process and checks whether it has completed.

After completion, the job is removed from the table.

This demonstrates basic job tracking and process-state management.